

An End-to-End AI Agent for Kaggle Competitions

Benchmarked against two frozen yardsticks on 20 Kaggle competitions

Wei-Hao Huang

Abstract

We present **my-agent**, an end-to-end agent that runs a machine-learning competition from raw data to submitted prediction with no person in the loop: an LLM reasons and decides at every stage — problem identification, exploratory analysis, feature engineering, modelling, evaluation, submission — while gradient-boosting libraries are invoked through the shell and a tree search over complete candidate solutions serves as the optimisation loop. We benchmark it against two frozen, methodologically distinct reference agents — **AIDE** (LLM tree search) and the **NVIDIA Kaggle Agent** (public-kernel reproduction) — on 20 Kaggle competitions, plus three mid-complexity competitions spanning language, physiological time series and medical imaging. Because an agent’s own cross-validation cannot arbitrate between agents, every verdict is a real submission scored on the competition’s live leaderboard, and every comparison is adjudicated by an error model read off Kaggle’s own two-way split of each test set. **my-agent leads the standing, winning 16 of its 24 decided duels (66.7%) against 42.3% for each reference agent**, taking the best private score on 10 of the 20 competitions and beating its own development-lane score on 13 of 20; on the three mid-complexity competitions it takes the three-way best on all three. The same error model refuses to call 22 of the 60 duels at all, and the agent’s two worst results fall on the two competitions with a mature public solution, where faithful reproduction beats autonomous search. One architectural finding follows from the same measurements: knowledge injected downstream, among the models being blended, was worth about $\sim 10^{-5}$, so this architecture injects it upstream at problem identification instead — and no local signal we tested can select the *form* that injection should take.

1 Introduction

Machine-learning competitions are *scorable tasks*: a submission receives a single measurable quality score on a hidden test set, and a public leaderboard records how it compares against thousands of human teams. Solving them end-to-end has traditionally demanded expert labor and extensive trial and error. LLM-driven coding agents have recently begun to automate this loop: AIDE performs LLM-guided tree search over candidate solutions [2]; ERA couples an LLM rewriter with tree search and externally injected research ideas, outperforming strong baselines across 16 Kaggle Playground competitions [1]; and industrial agents skip search altogether by reproducing the strongest public solution to a given competition [3].

This report is about one such agent, **my-agent**, built over the course of this internship: a hybrid design in which an LLM reasons at every stage of the competition pipeline, Auto-ML libraries are invoked through the shell for what they do best, a cross-competition experience library supplies evidence-backed priors, and a tree search over candidate configurations serves as the optimisation loop (Figures 1 and 2). It also documents itself: 22 five-section ML specification reports for my-agent and 17 re-documenting the AIDE runs are attached (Appendix A).

What the agent is worth is an empirical question, and this report answers it by measurement rather than by argument: 20 Kaggle competitions, two frozen reference agents, every verdict a real submission scored on the live leaderboard. my-agent leads that standing, winning 16 of its 24 decided duels (66.7%) against 42.3% for each reference agent, and takes the best private score on 10 of the 20. The same measurement refuses to call 22 of the 60 duels at all, and it locates the agent’s two worst results precisely: both fall on competitions with a mature public solution, where faithful reproduction beats autonomous search.

One design decision inside the agent earned its own experiments, because our first version got it wrong. Knowledge injected downstream, among the models being blended, measured about $\sim 10^{-5}$; the architecture therefore injects it upstream, at problem identification, through a stage that classifies task family, train–test window relation, split policy and external-data need before any model is fitted, emitting typed operators into a whitelisted, leakage-guarded external-data module.

All results are stated against two frozen, methodologically distinct yardsticks: **AIDE**, LLM tree search with no cross-competition memory, and the **NVIDIA Kaggle Agent**, which reproduces the highest-voted public kernel. All three run on 20 Kaggle competitions plus three mid-complexity competitions; because local estimates cannot be trusted to arbitrate between agents, every verdict among the 20 is scored on the real leaderboard via late submission, while the three mid-complexity competitions are graded offline and adjudicated under the error model of Section 3. my-agent’s benchmark lanes run under the frozen architecture in sandboxes (Section 3.1); all 20 are complete, audit-adjudicated and scored by a separate credentialed operator step, so the benchmark tables carry real my-agent leaderboard scores and paired verdicts throughout (Sections 4.2 and 4.4).

2 Methods

2.1 The Agent

The method is implemented as a Claude Code Skill (`kaggle-agent`) around a division of labor: the LLM (Claude Code) handles problem understanding, validation-strategy design, creative feature engineering, result interpretation and the choice of next step, while LightGBM / XGBoost / CatBoost / Optuna, invoked via the shell, handle model selection, hyperparameter optimization and stacking/ensembling. Execution follows the pipeline of Figure 1, whose **Stage 0.5** problem dossier decides what kind of problem the task is before any modeling (Section 2.2) and is evaluated in its own right in Sections 5.1 and 5.2. Modeling and evaluation run in one of two optimization modes:

- **Linear iteration protocol:** propose a hypothesis, run the experiment, record the result — used for the first pass and for small competitions.
- **Tree search (preferred):** once the linear protocol has produced a baseline single model and at least one blend, the agent switches to tree search. Internal evaluation shows tree search winning 9 of 10 competitions with 1 tie and 0 losses.

Three cross-cutting mechanisms support the pipeline (Figure 1):

Cross-competition experience library (`knowledge/experience.md`): every insight must carry an evidence field (`competition`, `exp #N`, `score A` $\rightarrow$ `score B`); entries without a score delta are rejected. The agent queries the library before EDA and before modeling, and writes validated new insights back after experiments. This is the most fundamental design difference between my-agent and the two reference methods.

Experiment records and external data. Every experiment’s parameters, features, split scheme, and scores are logged in structured form (`experiments.json`, `experiments_tree_v3.json`), making every number in this report traceable; the external-data layer realizes the dossier’s typed operators as whitelisted, leakage-guarded joins raced against the no-external baseline inside the search (Section 2.2); and each lane records a structured EDA pass (distributions, drift, correlation) before modelling, which the skill’s reporting layer renders, with the dossier and experiment records, into a per-competition ML specification report whose every printed number must trace to a logged experiment (Appendix A).

Tree-search protocol. Every run in this report uses the tree-search harness of Figure 2 (`tree_search/harness_v3.py`); the earlier downstream variant of that harness, whose measured near-null effect is what motivated the upstream layer (Section 2.2), was used by no benchmark submission here.

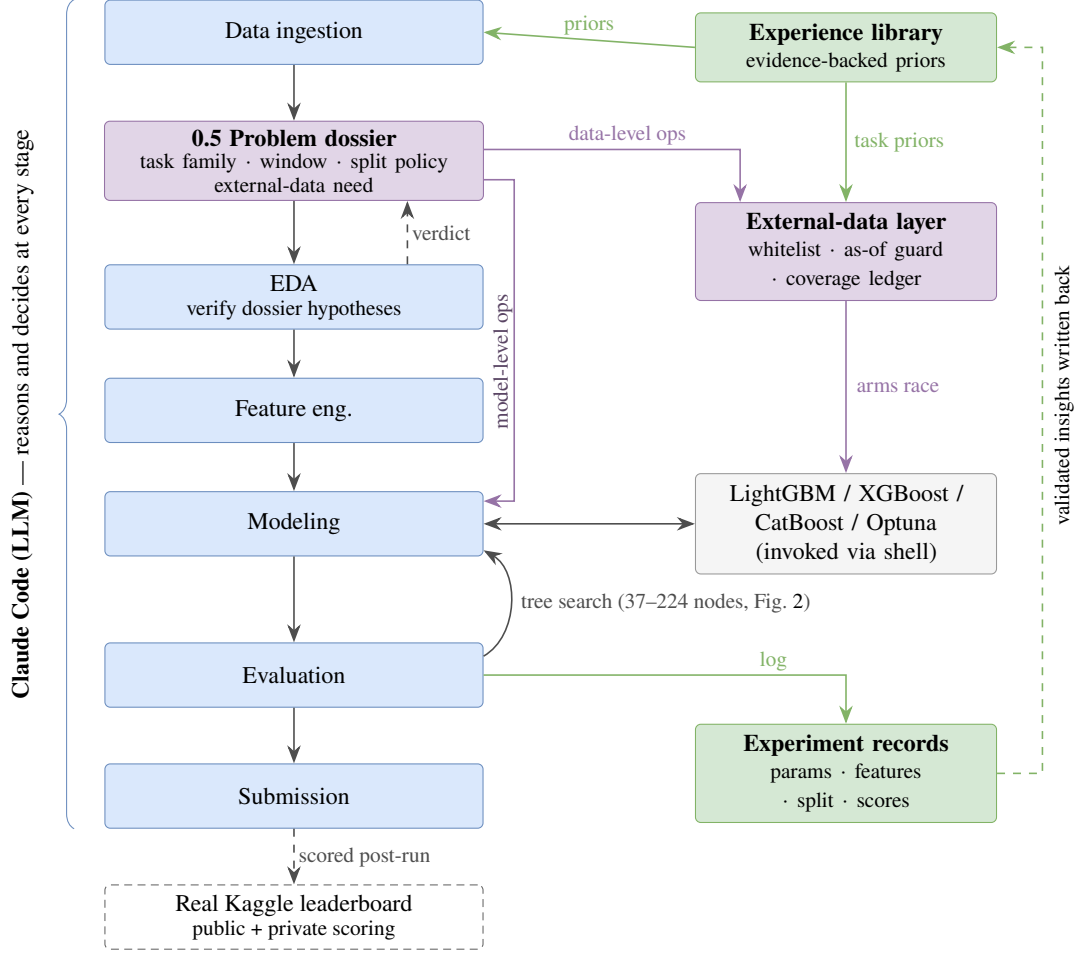


Figure 1: Architecture of my-agent, with color marking role. **Blue — the LLM-driven pipeline:** Claude Code reasons and decides at every stage, and EDA records a verdict on the dossier’s hypotheses (the dossier is a prior, not a conclusion). **Purple — the injection layer** (Section 2.2): the Stage-0.5 dossier emits typed operators, data-level ones into the external-data layer’s arms, which race a no-external baseline with the winner taking the full budget, model-level ones directly into search lineages. **Green — the evidence loop:** the experience library and the experiment records, with validated insights written back. **Gray — Auto-ML tools** invoked via the shell. Dashed arrows mark feedback, advisory or post-run steps. The leaderboard is external to the agent: lanes hold no credentials, and submissions are scored only after the run.

2.2 The Injection Layer

The purple elements of Figures 1 and 2 form a single layer, and its placement carries the design argument. An earlier downstream variant injected external ideas at the blend stage and measured $\sim 10^{-5}$; the architecture moves the injection point upstream to problem identification, where the question of what kind of problem this is gets decided.

Stage 0.5 — problem dossier. Before EDA, the agent answers “what kind of problem is this?”: task family, train–test window relation, split policy, and external-data need, written to a structured dossier. json. Judgments are grounded in `knowledge/task_priors.md`, a library of task-level priors each carrying an evidence field from our own benchmark. Reading competition-specific discussions or kernels is forbidden — the dossier must derive from the problem statement plus task-*type* knowledge only, keeping the method distinct from reproduction-based agents. The dossier is a prior, not a conclusion: EDA must verify its hypotheses and record a verdict.

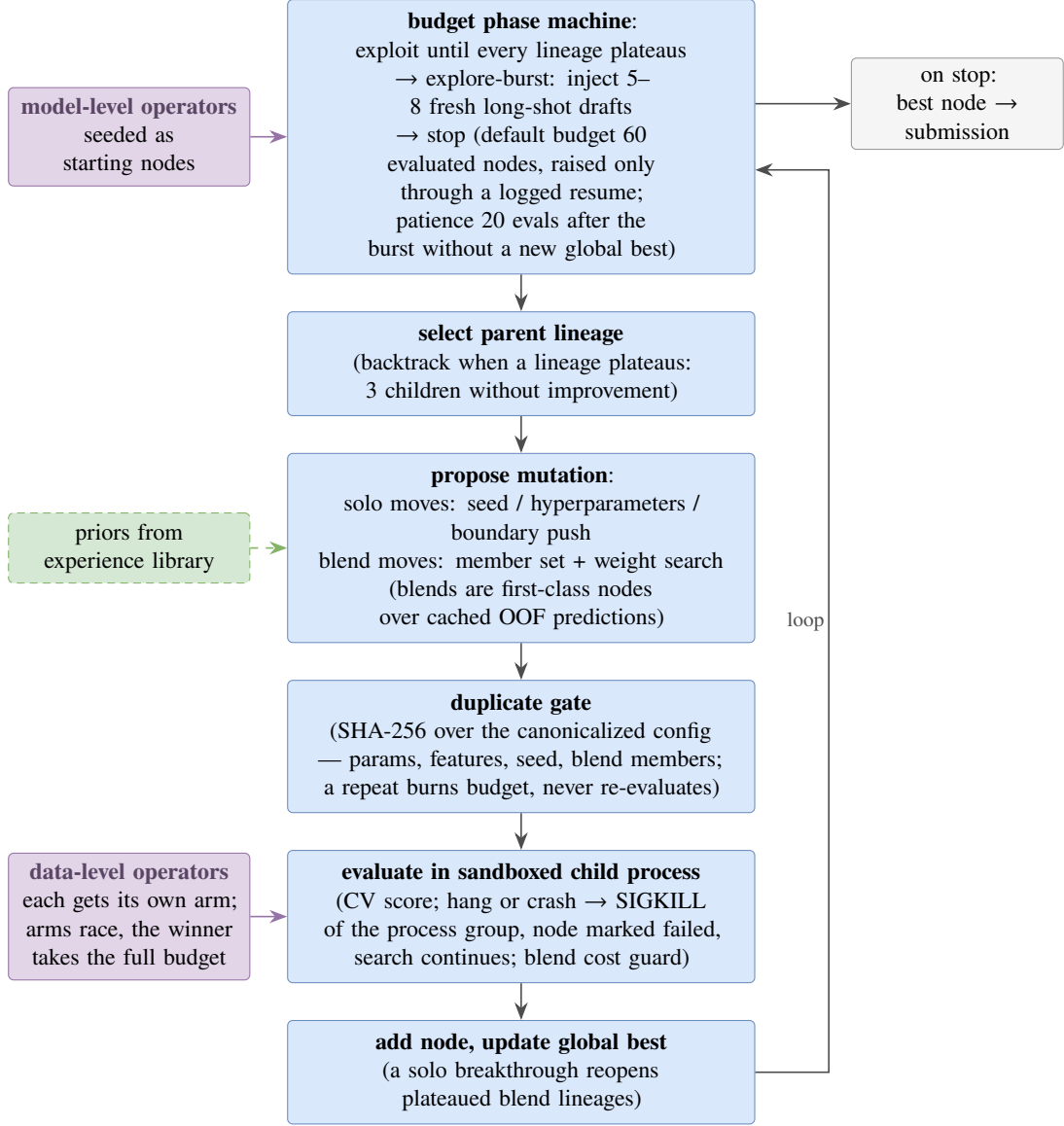


Figure 2: The tree-search loop used in all my-agent runs of this report (37–224 nodes per competition; per-competition counts in Table 8); Figure 3 shows what it builds. Each node is a complete, executable solution scored by local cross-validation. Purple: the injection layer’s operator entry points (Section 2.2); green: priors from the experience library; dashed arrows: advisory input and backtracking.

External-data module. A whitelist of public sources, joined through leakage-guarded as-of merges: an external value may reach a prediction row only if dated at or before the row’s period minus a declared lag. The module was adversarially verified, and the findings it reproduced — including silent-future-leak classes — were fixed under regression tests.

A typed contract between the two layers. The first version of this stack failed instructively: the dossier proposed World Bank GDP for s3e19 *as a country-level scale covariate*, but the execution layer could express exactly one transformation — appending a column to the feature list — and since a gradient-boosted tree cannot extrapolate a feature beyond its training range, the semantics evaporated between the layers with nothing raising an error. Both layers now read one closed vocabulary of typed operators (knowledge/injection_operators.md), and the dispatcher writes a coverage ledger, so a capability the execution layer cannot realize leaves a trace instead of resembling success (Appendix A.4).

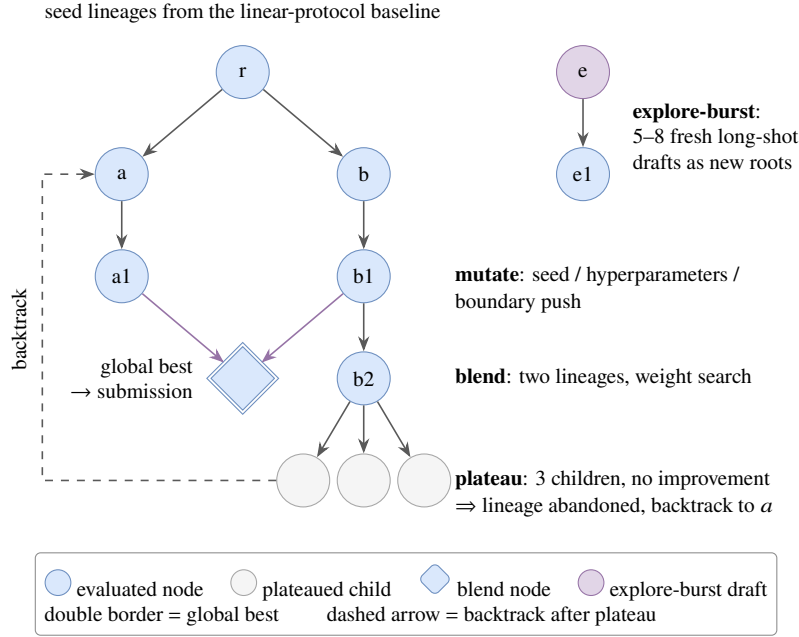


Figure 3: What the loop of Figure 2 builds: one search tree, schematic. Two lineages grow from the linear-protocol baseline, one plateaus after three children fail to improve and is abandoned, and a blend node over the cached out-of-fold predictions of $a1$ and $b1$ becomes the global best that gets submitted. Real runs evaluate 37–224 nodes per competition (Table 8).

2.3 The Two Yardsticks

AIDE (open-source agent, gen-1) performs LLM-driven tree search: at each step it writes a complete solution, executes it, reads the error or score, and mutates accordingly. It carries no cross-competition memory — every competition starts from zero.

NVIDIA Kaggle Agent (skill) reproduces the strongest public kernel: it researches the competition context, selects the highest-voted solution-type kernel, ports it faithfully, and retrains on local data.

Both reference methods are frozen — no improvements of any kind, even where we found weaknesses (the NVIDIA skill even ships a score-based selection tool we deliberately left disabled) — since improving a yardstick mid-study would make earlier and later data incomparable and would quietly turn the benchmark into an optimization of the reference methods.

3 Measurement Methodology

Two pieces of apparatus underlie every comparison below: a fairness and execution-discipline protocol that makes a score attributable to a method rather than to an accident of the shared machine, and an error model that says which score differences are readable at all. The offline-graded mid-complexity competitions of Section 6 are the one exception to the error model’s reach.

3.1 Fairness Protocol

The three agents share one machine, so four isolation conditions are mechanically enforced rather than left to convention (Table 1).

A fifth rule governs the experimenter rather than the agents: when the environment is defective, fix the environment, never hint to the affected side.

Benchmark execution discipline. For my-agent’s 20 benchmark lanes the protocol is tightened further. A benchmark manifest (`docs/rerun_manifest.json`) fixes exactly one workspace, one canonical eval-

Table 1: The four isolation conditions and what enforces each.

Condition	Rule	Enforcement
Data	Each side reads only its own root, containing files validated against the official Kaggle manifest	<code>build_isolated_roots.py</code>
Workspace	The three output directories are fully separated; each side is scored independently	Directory structure
Instruction	Prompts may carry environment facts and metric thresholds only — no modelling recipes, no other side’s scores, no workaround discovered by another side	Manual protocol + incident log
Resource	Only one lane executes at any moment	<code>lane_lock.sh</code> mutex

uator, and one search driver per competition; each lane executes in an isolated root, holds *no credentials* (the Kaggle token is stripped from the environment, so a lane can neither consult the leaderboard nor submit), and sees only its own workspace. A *per-lane transcript audit* then reads every tool call the lane made, and the launcher halts the whole queue after two consecutive audit failures, on the reasoning that two in a row indicts the pipeline rather than one lane.

One version per table. An architecture change invalidates every lane that ran before it, so the benchmark is versioned rather than patched: the run root is built once, its frozen file surface recorded in a manifest whose own hash is stored *outside* the root where no lane can reach it, and a clean-slate gate checks both before the queue starts. When the agent changed — most consequentially when the external-data layer was added — the in-flight run was archived and the benchmark restarted from a freshly built, freshly hashed root. Seven such partly-finished runs are archived beside the accepted one; no table in this report mixes lanes from two versions.

3.2 The Paired Test

Why an error model is needed at all. In the development-lane leaderboard measurements this apparatus was built on, several top-two gaps sat at the fourth or fifth decimal place (s5e10: 0.00003; s6e2: 0.00000); at that scale a report that declares winners without an error model is reporting its own noise back to itself.

How — the error scale comes from the data itself, not from assumptions. Kaggle randomly splits each test set into two disjoint subsets, public and private, and one submission receives one score on each. These are two independent estimates of the same model, so their difference is that competition’s metric sampling noise.

A failed first attempt, worth recording. We first used a single agent’s own public–private gap as the threshold, and 8 of 10 competitions came out as ties: that gap is mostly *shared* across the three agents — on s5e10 a common shift of 0.00025 against mutual scatter of only 0.00002 — so the paired construction subtracts it out.

The correct construction. Measure how much the *gap between two agents* moves across the two subsets — the common drift cancels in the subtraction. Writing d_{pub} and d_{priv} for the between-agent score difference on the public and private subsets, a verdict requires both:

$$\text{sign}(d_{\text{priv}}) = \text{sign}(d_{\text{pub}}) \quad (\text{the ordering replicates on disjoint samples}) \quad (1)$$

$$|d_{\text{priv}}| > |d_{\text{priv}} - d_{\text{pub}}| \quad (\text{the gap exceeds its own movement}) \quad (2)$$

Statistical properties of the decision rule. Condition (2) is algebraically equivalent to $d_{\text{pub}}(2d_{\text{priv}} - d_{\text{pub}}) > 0$ and therefore *implies* condition (1); both are stated because they diagnose different failures. The rule is a conservative screen rather than a calibrated hypothesis test — under a null of true equivalence it still returns spurious verdicts at a computable, non-negligible rate — and it adjudicates every leaderboard verdict in this report, including the benchmark duels of Section 4.4.

The second error term: between-run agent variance. The paired test covers test-set sampling noise only; agent execution is itself stochastic. On the four competitions with the tightest development-lane duels, my-agent ran twice fresh under identical conditions and AIDE’s pinned re-run pairs against its frozen baseline. Private-leaderboard spreads:

	my-agent	AIDE	development-lane duel gaps
s4e11	0.00021	0.00027	0.00013–0.00033
s3e16	0.00460	0.00056	0.00142–0.00365
s5e10	0.00000	0.00003	0.00003–0.00012
s6e2	0.00000	0.00020	0.00000–0.00008

Two readings. The variance is substantially a property of the *task*, not the agent — both agents agree s3e16 is the noisy competition by an order of magnitude — so error bars are per-competition rather than one global number, and gaps at or below a measured spread (s5e10’s 0.00003 exactly equals AIDE’s wobble there) are undecidable: the floor the benchmark verdicts of Section 4.4 apply. NVIDIA’s variance term remains unmeasured (Section 7.5).

4 The Three-Way Benchmark

4.1 Design

Evaluation set. 20 tabular competitions (Table 2), each providing both a public and a private test-subset score — the inclusion criterion the paired test requires (Section 3.2). Four further competitions the three agents ran are excluded: no private score can be obtained for them (late submission closed, rolling leaderboards).

Execution conditions. The three agents ran serially under the mutex of Section 3.1. AIDE: 20 steps, its own default budget (conway stopped at 16 due to the 4-hour cap). NVIDIA: reproduced public kernels per its method. my-agent: the full six-stage pipeline including tree search (37–224 nodes per competition; Appendix A.5), each lane under the execution discipline of Section 3.1.

4.2 Raw Scores

Goal and procedure. Each side independently produces a submission, scored on Kaggle via late submission for public and private scores and a percentile rank (PR); the AIDE and NVIDIA columns of Table 3 are those frozen leaderboard scores. my-agent’s lanes hold no credentials, so nothing inside a lane ever touches the leaderboard: each lane freezes a `submission.csv`, and a separate credentialed operator step scored all 20, filling the LB column of Table 3; the percentile ranks are the one quantity not reported, because they require a leaderboard-snapshot pull we did not run. No local CV is quoted in this report’s tables, because it is never a leaderboard proxy (s3e19’s pooled OOF SMAPE of 6.71 sat against a realized lane score of 49.41).

PR is the share of that competition’s final private standings the submission would beat — an *absolute* capability reference, separate from the relative ordering among the three.

Table 2: Competition descriptions. Tasks fall into two families: classification (binary, ordinal; 7 competitions) and prediction/regression (incl. time series and multi-target; 13 competitions).

#	Competition	Task type	Train rows $\times$ cols
1	s3e1	Regression	$37,137 \times 10$
2	s3e3	Classification (binary)	$1,677 \times 35$
3	s3e5	Classification (ordinal)	$2,056 \times 13$
4	s3e7	Classification (binary)	$42,100 \times 19$
5	s3e9	Regression	$5,407 \times 10$
6	s3e11	Regression	$360,336 \times 17$
7	s3e14	Regression	$15,289 \times 18$
8	s3e16	Regression	$74,051 \times 10$
9	s3e19	Regression (time series)	$136,950 \times 6$
10	s4e11	Classification (binary)	$140,700 \times 20$
11	s5e10	Regression	$517,754 \times 14$
12	s6e1	Regression	$630,000 \times 13$
13	s6e2	Classification (binary)	$630,000 \times 15$
14	afsis-soil-properties	Regression (multi-target)	$1,157 \times 3,600$
15	cat-in-the-dat	Classification (binary)	$300,000 \times 25$
16	conway-s-reverse-game-of-life	Structured multi-output	$50,000 \times 802$
17	tps-aug-2022	Classification (binary)	$26,570 \times 26$
18	tps-jan-2022	Regression (time series)	$26,298 \times 6$
19	s5e1	Regression (time series)	$230,130 \times 6$
20	tps-sep-2022	Regression (time series)	$70,128 \times 6$

4.3 Execution and Audit

The queue ran under operator approval and finished with all 20 lanes complete and audit-adjudicated, halting three times on the way — twice on the same environment fault, once on its two-consecutive-audit-failures rule. That fault was ours rather than the method’s: the sandbox bound the session’s credential file read-only, so a lane whose access token expired mid-run could not write the refreshed token back and died; the post-benchmark fix binds it writable and refuses to start a lane whose token cannot outlive it. The audit gate was exercised in both directions, most sharply on **s3e19**, whose first attempt opened the raw knowledge library four times and was discarded. (It also tripped the network detector, but a re-scan with the corrected detector below finds no network command — the hit was a Python variable named `nc` in a diagnostic heredoc, and the four raw opens carry the discard on their own.) The clean re-run scores *worse* on local CV — pooled OOF SMAPE 6.706 against the tainted attempt’s 5.740 — consistent with the contamination advantage being real and the audit pricing it correctly; a benchmark that discards a better-looking number and keeps the honest one has evidence its audit does something. The remaining incidents turned on transcript ordering: on s5e10, tps-aug-2022 and tps-sep-2022 the flagged raw-library opens all fell strictly *after* the submission had been written and validated, so those scores stand (s5e10’s hand-edit of the shared library after finalisation was reverted before any later lane ran). Exactly one instance crossed lanes: tps-jan-2022 started two minutes after aug-2022’s write-back and its Stage 0.5 read served the polluted entries, so the operator killed that lane at one minute, reverted the library, and relaunched it clean — the re-run raised the same class of blocker once more, again only after its own submission was validated, and was accepted on the same ordering argument.

The recurrence is the finding, not the individual incidents. Four of the twenty lanes (s5e10, tps-aug-2022, tps-sep-2022, tps-jan-2022) wrote their own `library_writeback.py`, because the skill instructs every lane to write validated insights back through library tools this run root does not ship, and the root bound the shared library writable — a design fault of the harness rather than of any lane, and only one instance of it crossed lanes. The fix mounts `knowledge/` read-only over the writable run root, so

Table 3: The complete 20-competition benchmark. All cells are public/private leaderboard scores; **bold** marks the best *private* score of the three (raw, direction-aware; the paired-test verdicts, which this raw reading does not replace, are in Table 4). my-agent’s column comes from the credentialed operator scoring step; its percentile ranks await a leaderboard-snapshot pull. NVIDIA and AIDE cells are frozen leaderboard scores with percentile rank (PR) in parentheses. Lane status: all 20 lanes complete, audited and scored (~26 hours of accepted lane attempts); tps-jan-2022 is the clean re-run after its first attempt was discarded by the audit (footnote *h*).

#	Competition	Metric	my-agent LB (pub/priv)	NVIDIA LB (PR)	AIDE LB (PR)
1	s3e1	RMSE ↓	0.55715/ 0.55275	0.55332/0.55550 (93.2)	0.55899/0.55552 (81.0)
2	s3e3	ROC-AUC ↑	0.89449/0.87157	0.93526/ 0.89537 (66.8)	0.88484/0.86863 (26.4)
3	s3e5	QWK ↑	0.57041/0.57223	0.57515/0.56974 (75.7)	0.59362/ 0.58138 (79.8)
4	s3e7	ROC-AUC ↑	0.91127/0.90441	0.91230/ 0.90458 (78.1)	0.90934/0.90109 (67.1)
5	s3e9	RMSE ↓	11.84750/12.33192	11.83399/ 12.22913 (59.1)	11.86657/12.30667 (44.5)
6	s3e11 ^a	RMSLE ↓	0.29309/ 0.29368	0.29559/0.29624 (66.6)	0.29389/0.29445 (73.3)
7	s3e14	MAE ↓	343.43434/333.18611	338.36524/ 330.71616 (89.2)	342.44918/332.31556 (71.2)
8	s3e16	MAE ↓	1.33738/ 1.33822	1.33566/1.34001 (96.4)	1.33920/1.34224 (91.4)
9	s3e19 ^b	SMAPE ↓	49.11872/49.40878	47.52145/ 48.26558 (41.4)	48.39715/48.34131 (40.1)
10	s4e11	Accuracy ↑	0.94195/ 0.94107	0.94136/0.94076 (52.8)	0.94184/0.94056 (62.1)
11	s5e10 ^c	RMSE ↓	0.05554/ 0.05576	0.05562/0.05588 (59.4)	0.05555/0.05579 (78.9)
12	s6e1 ^d	Error ↓	8.68844/ 8.71516	8.70480/8.73239 (69.5)	8.69449/8.72191 (74.8)
13	s6e2 ^e	ROC-AUC ↑	0.95354/0.95505	0.95348/0.95508 (63.0)	0.95364/ 0.95516 (76.5)
14	afsis	MCRMSE ↓	0.41902/ 0.48917	0.61276/0.69855 (18.8)	0.45636/0.49861 (45.6)
15	cat-in-the-dat	ROC-AUC ↑	0.80800/ 0.80243	0.77641/0.77084 (32.6)	0.80790/0.80217 (70.3)
16	conway ^f	MAE ↓	0.10313/ 0.10414	0.11094/0.11189 (96.5)	0.10960/0.11065 (97.9)
17	tps-aug-2022 ^g	ROC-AUC ↑	0.58534/0.58930	0.57920/0.58730 (43.0)	0.58479/ 0.59106 (60.5)
18	tps-jan-2022 ^h	SMAPE ↓	4.58859/6.29895	4.43958/ 4.63651 (76.4)	10.11598/10.66638 (28.9)
19	s5e1	MAPE ↓	0.09076/ 0.08401	0.05726/0.14594 (69.3)	0.12573/0.17917 (48.1)
20	tps-sep-2022	SMAPE ↓	28.45070/28.49605	4.77458/ 5.48351 (80.3)	5.78998/9.72095 (48.3)
Median PR (range)			pending snapshot	68.0 (18.8–96.5)	68.7 (26.4–97.9)

^a First attempt exited without a submission; completed on a second attempt. ^b Clean re-run; first attempt discarded by the transcript audit (Section 4.3). ^c Audit incident resolved, score accepted (Section 4.3). ^d Completed and audit-accepted. ^e Second attempt; the first exited without a submission. ^f The scoring tool’s structural validator false-blocked conway’s file (sample-file naming); the operator submitted the identical file (same SHA-256) manually via the Kaggle CLI. ^g Submission validated before the lane’s post-submission library violations; accepted on that ordering (Section 4.3). ^h Clean re-run after the first attempt was discarded for cross-lane contamination; its own post-submission library violations are accepted on the same ordering argument as ^g (Section 4.3).

prevention no longer depends on after-the-fact detection. Re-reading the transcripts also exposed two defects in the auditor itself: its network detector matched the substring `kaggle` inside the path of the skill’s own submission helper (two lane audits blocked on that string alone, each costing a manual scan and a recorded waiver) and, more seriously, its exemption list dropped every command beginning with `uv` — which is every command a lane issues — so `uv run kaggle competitions leaderboard`, the exact command the detector exists to catch, was never examined. Both are fixed; re-running the corrected auditor over all 20 transcripts returns zero network findings and leaves the genuine library blockers unchanged. A detector blind at its own target is the kind of defect only a post-hoc re-read finds.

4.4 The Standing

The benchmark’s headline reading is the full duel matrix: 20 competitions $\times$ 3 pairings = 60 duels, each adjudicated by the rule of Section 3.2 plus the per-competition variance floors. With all 20 lanes scored, every one of the 60 duels is adjudicated: 38 produce a verdict and 22 are undecidable — the class the error model exists to catch, including a live replication of the development-lane `s5e1` case (the public half of that test split favours NVIDIA, the private half favours my-agent, so the sign test cannot resolve it) and three duels killed by the variance floors — two of them wins my-agent would otherwise have

claimed (s3e16, and s5e10, where the private gap of 0.00003 exactly equals AIDE’s measured spread — the case Section 3.2 flagged in advance; the rule requires the gap to *exceed* the floor, and we compare in exact decimal, so this duel stays undecidable rather than becoming a 17th my-agent win) and one a would-be AIDE win (s6e2, gap 0.00011 under AIDE’s 0.00020 floor). Table 4 gives the standing. my-agent leads outright: it wins 16 of its 24 decided duels (66.7%; each yardstick wins 11 of its 26, 42.3% — the denominators differ because a duel is decided or not independently for each pair), takes the best private score on 10 of the 20 competitions (NVIDIA 7, AIDE 3), and its rerun submission beats its own development-lane private score on 13 of 20. Its sharpest losses fall where the mechanism of Section 7.3 predicts they should: on tps-sep-2022, the one benchmark competition with a mature, high-quality public baseline, both yardsticks beat it decisively (5.48 and 9.72 SMAPE against 28.50), and on tps-jan-2022 NVIDIA’s reproduction of the known GDP-ratio solution beats it 4.64 to 6.30. Reproduction dominates exactly where a strong public solution already exists; search wins the rest.

Table 4: Standing under the paired test with per-competition variance floors, final state: all 60 duels adjudicated (38 decided, 22 undecidable). Win rate is over each agent’s own decided duels (my-agent 24, each yardstick 26); the 22 undecidable duels are not ties — 19 fail the sign or stability test and 3 die on a variance floor.

Method	Wins	Losses	Win rate (decided duels)
my-agent	16	8	66.7%
NVIDIA	11	15	42.3%
AIDE	11	15	42.3%

5 The Injection Layer Under Control

5.1 Does the Dossier Fire on the Right Tasks?

Design and result. Run on the 18 baseline competitions from the problem statement, data schema, and task priors alone — minutes per competition, no training — the dossier layer should fire only where external data is genuinely needed. It fires on exactly the two country-panel time-series competitions, s3e19 and jan-2022, whose accepted solutions are known to require external covariates, and reports “no / unlikely” on the other 16, which span iid-tabular, categorical, spectral and structured-output tasks. One caveat keeps this honest: the task priors were distilled from those two competitions’ own failure analysis, so the silence is the informative half.

5.2 What the Layer Delivers

Design. Across the firing class — s3e19, the held-out tps-sep-2022 and the pre-registered s5e1 — matched arms differ only in the operator’s added columns (verified digit-identical otherwise); every arm is submitted for real public and private scores and judged by the paired test of Section 3.2, and the s5e1 test was pre-registered. Full arm tables are in the companion case-studies document.

Effect. Across the three competitions, **all six external-data arms beat their no-external baselines, every one paired-significant**. On the canonical failure itself the ablation against a searched champion from the identical pipeline with the layer disabled passes the paired test, private 48.243 against 50.455 — both controlled arms, not the benchmark s3e19 lane, which scored 49.40878 (Table 3).

The operating rule the experiments imposed. One negative result ships with the layer: no local signal we tested selects which *form* the injection should take. Cross-validation, a held-out-year protocol, and a proportionality diagnostic each misrank the form verdict (0-for-3); the diagnostic — the one selector that survived hindsight — was put to a pre-registered test on s5e1, its prediction committed to version

control before any model was fitted, and its named falsifier fired. The layer therefore *races both forms inside the search, always*, and the one surviving post-hoc candidate (horizon length) is committed as the H-HORIZON pre-registration for the next firing-class competition (Section 7.5).

What the judgment layer proposes, and what actually consumes it. The layer’s coverage ledger overstated itself: operators were emitted into a key no search driver read while the dispatcher recorded them as realized. A produced artifact with no consumer is indistinguishable from a working one in every log, score table and coverage metric — inside the very mechanism built to prevent silent truncation (Appendix A.4). **An architecture claim needs a consumer trace, not a vocabulary count.**

6 Beyond Tabular Competitions

Design. The three agents run, each under its frozen method and the same isolation protocol, on three competitions outside the tabular Playground family: us-patent (NLP text pairing), ventilator-pressure-prediction (physiological time series) and SIIM-ISIC melanoma (dermoscopic imaging). None of the my-agent lanes ran the injection layer or tree search (Section 7.4) — a node here is a full deep-learning fine-tune, not a seconds-long GBDT refit — so all three took the skill’s documented fallback: one linear pass to a validated single model plus a blend.

Scoring. The mid-complexity competitions use MLE-bench [4] prepared splits and are graded offline against the held-out private split; medal thresholds and the above-median flag come from the frozen leaderboard snapshot.

Table 5: Mid-complexity three-way, offline-graded against the held-out private split. Bold = three-way best; † = above the frozen leaderboard’s median.

Competition	Metric	my-agent	NVIDIA	AIDE
us-patent	Pearson r ↑	0.86658 † (silver)	0.86160† (bronze)	0.65821
ventilator	MAE ↓	0.16525	0.40093	0.34591
SIIM-ISIC	AUC ↑	0.93529 †	0.89822	0.88579

Results. my-agent takes the three-way best on all three and the only above-median score on SIIM — a clean sweep across three unrelated task families, so the result is about pipeline discipline rather than tabular-specific tuning.

7 Discussion

7.1 Contributions

1. **Placement against two frozen, methodologically distinct yardsticks, under a rule committed before the scores existed.** All 20 lanes audited and scored on the real leaderboard: 16 wins to 8 losses, 66.7% of my-agent’s decided duels against 42.3% for each yardstick (Section 4.4), plus the three-way best on all three mid-complexity competitions.
2. **An end-to-end agent whose knowledge enters at problem identification, not at blending** (Sections 2.2 and 5.1–5.2): 2-of-18 dossier separation, six of six external-data arms paired-significant, and the negative result that no local signal selects the injection’s *form*, the one selector surviving hindsight killed by its own pre-registered falsifier.
3. **A ranking is a property of method × evaluation set**, demonstrated rather than asserted and applied to our own agent first (Section 7.3).

7.2 Comparing the Three Agents

The differences lie not in tuning skill but in where knowledge comes from and how the next step is decided (Table 6).

Table 6: Design comparison of the three methods.

Aspect	my-agent	AIDE	NVIDIA
Knowledge source	Cross-competition experience library (accumulating)	None (from zero each time)	Public kernels fetched fresh each time
Search strategy	Six-stage pipeline + tree search (37–224 nodes)	Tree-shaped trial and error, 20 steps	No search; faithful reproduction
Basis for next step	Experience-library priors + OOF diagnostics	Previous step’s error or score	Highest-voted kernel
Validation rigor	Split chosen by task type; diagnostics tagged and excluded	No concept of split legality	Inherits the kernel’s split
External data	Injection layer (typed operators, whitelisted sources)	Never considered	Inherited from the kernel

The yardsticks’ wins track their knowledge source — reproduction where a mature public solution exists, search depth where none does (Section 7.3). my-agent’s edge is validation rigor and accumulated knowledge — it quarantined the same shuffled split AIDE submitted — while its measured bottleneck was judgment at problem identification, the gap the injection layer was built to close; form selection there remains open (0-for-3).

7.3 A Ranking Belongs to Method $\times$ Evaluation Set

The most important methodological lesson of the development runs: swap the competition set and the paired-test winner flipped between a Playground-only subset and the broader benchmark set (those standings belong to superseded development lanes and are not reprinted here; the mechanism is a property of the methods and survives). Reproduction dominates where a mature public solution exists, autonomous methods dominate on obscure, build-your-own-pipeline tasks, and relative verdicts binarize many near-tie gaps, so the paired win rate is highly sensitive to sample composition even while absolute position (median PR) stays stable. The s5e1 duel of Section 4.4 reproduces the caution inside one competition: the public and private halves of a single test split order two agents oppositely.

7.4 Limitations

- **The significance test covers only test-set sampling noise**, not between-run agent variance. That variance is measured for my-agent and AIDE on four competitions (Section 3.2) and enters the error model as a per-competition floor; NVIDIA’s term remains unmeasured.
- **The duels are not independent trials.** Each competition contributes three duels from six shared scores, so the effective sample size is closer to 20 competitions than to 60 duels. And reproduced kernels were tuned by their authors against the realized public leaderboard, so exchangeability is weakest exactly where the reproduction method is strongest.
- **Budgets are per-method conventions, not compute-matched:** AIDE’s default 20 steps, my-agent’s 37–224 evaluated nodes plus a linear pass, NVIDIA’s minutes of reproduction — each method “as shipped.”
- **The sample is small, sensitive to its composition, and partly added post hoc.** Removing one competition moves a win rate by several points — dropping s3e19 alone takes my-agent from 66.7% to

72.7%; dropping s3e3 takes NVIDIA from 42.3% to 37.5% (Section 7.3). The two firing-class competitions joined after their leaderboards had been consulted for the injection experiments — disclosed, not hidden.

- **The experience-library asymmetry is disclosed, not eliminated:** my-agent carries cross-competition memory; the other two cold-start.
- **The mid-complexity lanes ran without the injection layer and without tree search** (Section 6), and offline grading yields one score per submission, outside the paired test’s reach.

7.5 Open Problems

Operator form cannot be selected from any local signal we have tested — 0-for-3, with one selector killed by pre-registration. The one surviving candidate, horizon length, is post-hoc; it is committed as the H-HORIZON pre-registration, to be tested on the next firing-class competition before its leaderboard is consulted. This is the sharpest open question here.

The firing class is small , so the controlled evidence rests on two clean test points plus one held-out replication; extending it is worth more than re-running the competitions where the layer stays silent, which changes nothing the layer does.

The experience library’s residual weak link is retrieval quality. Enforcement is closed — an experiment record without a `library_query` trace is invalid, checked mechanically by the audit tool. Quality is not: a neighbouring entry over-claimed and had to be corrected against leaderboard evidence, and no mechanism yet scores retrieved priors for applicability. Smaller items: whitelisting further source classes (prior-competition archives, public text corpora), and a direct ablation of the experience library.

8 Conclusion

The benchmark verdict is complete: all 20 transcript-audited lanes were scored on the real leaderboard and adjudicated by the error model committed in advance (Section 3) — my-agent leads with 16 wins to 8 losses, 66.7% of its decided duels, while 22 of the 60 duels stay undecidable at the measured noise scale. The architectural answer is placement: downstream injection measured near-null, so knowledge enters at problem identification, and the layer’s controlled, leaderboard-scored evidence carries the claim (Section 5.2). Two negative results ship with it and count as the method working: no local signal selects the injection’s form, the one selector surviving hindsight killed by its own pre-registered falsifier; and the audit discarded a contaminated lane whose tainted score looked better than its clean replacement’s. A benchmark that can kill its own hypotheses is the thing this study set out to build.

A Experimental Procedure

A.1 Per-Competition Specification Reports

A specification report accompanies this paper for every agent on every competition, written to the five sections of Tso-Jung Yen’s ML specification framework — Data / Models & Architecture / Training / Inference / Evaluation & Benchmarking — and archived at [docs/ml_specs/](#) for my-agent and [benchmark_results/ml_specs_aide/](#) for AIDE, in Markdown and PDF.

Every number in these reports is extracted deterministically by `collect.py/collect_aide_facts.py` from that competition’s structured records (`facts.json`, `eda_summary.json`, `experiments_tree_v3.json`, `journal.json`), never generated by an LLM; unavailable fields are marked “not recorded” rather than left blank or estimated. The framework’s deep-learning-specific fields — learning-rate schedule, batch size, transfer learning, data augmentation — are marked not applicable throughout,

in these reports, whose pipelines are all gradient-boosted-tree based. The reports are CV-only by construction; their conclusions must not be mixed with leaderboard verdicts.

A.2 Split Strategy by Task Type

Validation-split policy is fixed by task type in every lane, and shuffled KFold on time-series data is forbidden (Table 7).

Table 7: Validation-split policy by task type.

Task type	Required split	Rationale
Time-series extrapolation (test set later than training period)	Time-based split (<code>year < y</code> vs <code>year == y</code>) or <code>TimeSeriesSplit</code>	Each validation block must be strictly later than its training window, mirroring the true train–test gap
Text pairing (same anchor in many rows)	<code>GroupKFold</code> by anchor	Otherwise one anchor spans train and validation, and the model memorizes anchors instead of semantic relations
Generic tabular (continuous i.i.d. target)	Shuffled <code>KFold</code> / <code>StratifiedKFold</code>	No temporal or group structure

A.3 Execution Environment

All lanes ran on ARM64 Linux (Ubuntu 24.04) with an NVIDIA GB10 GPU, under a uv-managed Python 3.13 environment whose versions are frozen in `uv.lock`. Reproducing a score also requires the determinism settings the lanes ran under: fixed seeds throughout, and LightGBM configured with `deterministic=true`, `force_row_wise=true` and a fixed `num_threads`, without which the same configuration does not reproduce across processes.

A.4 Companion Documents

Two companion documents accompany this report. *Building the Injection Layer: An Engineering Log* carries the per-regime coverage tables behind the consumer-trace finding of Section 5.2, together with the audit scripts that reproduce them and the minute-by-minute reconstruction of the voided ventilator attempt. *Per-Competition Case Studies* carries the single-competition forensic depth behind the condensed claims of Section 5.2: the CV–leaderboard reversal mechanics, AIDE’s step-by-step split switch, the s5e1 setting behind the pre-registered prediction, and the s3e19 residual decomposition in full.

A.5 Tree-Search Size per Competition

Table 8: Evaluated tree-search nodes per competition (my-agent), from each completed benchmark lane’s search records (`experiments_tree_v3.json` / `lane STATUS.md`). The default budget is 60 evaluated nodes; a lane could extend it through the logged resume path. `s5e1`’s 224 is the sum of its separately raced per-arm trees; the largest single tree is `tps-aug-2022`’s 126; `conway` stopped at 37 on the wall-clock backstop.

Competition	Nodes	Competition	Nodes	Competition	Nodes
s3e1	53	s3e16	45	s6e1	47
s3e3	60	s3e19	60	s6e2	57
s3e5	60	s4e11	110	afsis	60
s3e7	49	s5e1	224	cat-in-the-dat	70
s3e9	107	s5e10	46	conway	37
s3e11	60			tps-aug-2022	126
s3e14	70			tps-jan-2022	81
				tps-sep-2022	60

B Data and Code Availability

All my-agent code and every benchmark artifact are archived in the project repository github.com/930731haohao-afk/kaggle-agent (private; access granted on request): the agent skill definition and its per-stage references, the tree-search harness, the experience library, and the injection layer, which lives in `external_data/` and `tree_search/`. Infrastructure scripts that sit at `ai_agents/` level in the working tree are mirrored under `benchmark_infra/`, and the cross-agent artifacts — the three-way score master table, the benchmark-run manifest, the coverage status table, and the AIDE specification reports — under `benchmark_results/`, with my-agent’s specification reports and per-competition experiment logs alongside them in `docs/ml_specs/` and `competitions/`. The repository also carries the drivers that rebuild the isolated data roots and re-run all three agents on a single competition. Every number in this report is traceable to those structured records.

References

- [1] Aygün, E., et al. An AI system to help scientists write expert-level empirical software. *Nature* (2026). <https://doi.org/10.1038/s41586-026-10658-6>
- [2] Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138 (2025).
- [3] NVIDIA. Kaggle Agent: kernel-reproduction agent skill (software), 2026.
- [4] Chan, J. S., et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. arXiv:2410.07095 (2024).